



Instituto Politécnico
de Viana do Castelo

Programação I

Algoritmos de ordenação e pesquisa



Algoritmos de ordenação:

- BubbleSort
- SelectionSort
- InsertionSort
- QuickSort

Algoritmos de pesquisa:

- Sequencial
- Ordenada
- Binária

Ordenação



Um algoritmo de ordenação coloca os elementos de uma dada sequência numa ordem específica. As ordens mais usadas para as ordenações são a **ordem numéricas** e a **ordem alfabética**. Outras formas são:

- Ordenação por **data**: em que elementos contendo informações de data e hora são ordenados em ordem cronológica;
- Ordenação por **estrutura**: em que elementos que possuem campos com diferentes tipos de dados são ordenados com base em um ou mais campos específicos;
- Ordenação por **bit**: em que elementos são ordenados com base em seus valores binários;
- Ordenação por **frequência**: em que elementos são ordenados com base em sua frequência de ocorrência em um conjunto de dados;
- Ordenação por **tamanho**: em que elementos são ordenados com base em seu tamanho em bytes;
- Ordenação por **proximidade**: em que elementos são ordenados com base em sua distância em relação a um valor de referência.

Ordenação



A principal razão para se ordenar uma sequência é o aumento da velocidade de acesso aos dados.

Os algoritmos de ordenação podem ser classificados em dois tipos:

- Internos – ordenação de informação armazenada em sequências (arrays);

val[0]	val[1]	val[2]	val[3]	val[4]	val[5]	val[6]	val[7]
70	25	34	15	5	60	3	55

- Externa – ordenação de informação armazenada em ficheiros.



Os algoritmos de ordenação devem ser eficientes quer em termos de **velocidade** de **execução** como em termos de espaço ocupado.

Existem vários algoritmos de ordenação:

- Uns de implementação simples, outros de implementação mais complexa.
- Alguns algoritmos de ordenação (os mais simples) ordenam os elementos no seu próprio vetor (array), fazendo para isso um rearranjo interno dos seus elementos. Outros usam vetores auxiliares.
- Uns mais eficientes para sequências de grandes dimensões, outros mais eficientes para sequências de pequenas dimensões.

Ordenação

Os algoritmos de ordenação mais usados são:

- Bubble sort
- Selection sort
- Insertion sort
- Quick sort
- Shell sort
- Merge sort
- Radix sort
- Shaker sort

How many shortest-length paths are there to get from your house to the doughnut shop?

$\binom{n}{k} = \frac{n!}{(n-k)!k!}$

$e^{i\pi} + 1 = 0$

$\binom{11}{7} = \binom{11}{4} = 330$ paths

Onto

One-to-One

There are six dogs to give 13 tacos. Use a 'stars and bars' diagram to illustrate the first and sixth dog get 3 tacos, the second dog gets none, the third dog gets 5 and the fourth dog gets one.

$A = \{2, 4, 10, 15\}$

$(A \cup B \cup C) \cup (A \cap B \cap C)$

$v = e + f = 2$

P.I.E. Example:

$6! - \left[\binom{6}{1}5! - \binom{6}{2}4! + \binom{6}{3}3! - \binom{6}{4}2! + \binom{6}{5}1! \right]$

Find $7 + 12 + 17 + 22 + \dots + 342$

$S_n = 7 + 12 + 17 + 22 + \dots + 342$

$+ S_n = 342 + 337 + 332 + 327 + \dots + 7$

$2S_n = 349 + 349 + 349 + 349 + \dots + 349$

$2S_n = 349 \cdot 68$

$S_n = \frac{349 \cdot 68}{2}$

$S_n = 11866$

Original: $\exists x \forall y (x \geq 2y \rightarrow x > y + 1)$

Converse: $\exists x \forall y (x > y + 1 \rightarrow x \geq 2y)$

Negation: $\neg [\exists x \forall y (\neg (x \geq 2y) \vee x > y + 1)]$

$\forall x \exists y (x \geq 2y \wedge x \leq y + 1)$

Contrapositive: $\exists x \forall y (x \leq y + 1 \rightarrow x < 2y)$

Ordenação



A descrição dos algoritmos de ordenação que usam a própria sequência para a ordenação assentam nos seguintes pressupostos:

- A entrada é um vetor (array) cujos elementos precisam ser ordenados
- A saída é o mesmo vetor com os seus elementos ordenados
- O espaço que pode ser utilizado é o espaço do próprio vetor, por vezes, com auxílio de **variáveis temporárias auxiliares**.



Bubble sort

Bubble sort ou ordenação “bolha”, é um algoritmo de ordenação simples. O seu nome deve-se ao facto de os elementos maiores subirem para as suas posições corretas formando uma imagem tipo “bolha”.

A estratégia do algoritmo é a seguinte:

- Percorre-se o vetor da esquerda para a direita comparando os elementos consecutivos e trocando os elementos que estão fora de ordem.
- Desta forma garante-se que o elemento com maior valor será levado para a última posição do vetor.
- Repete-se o processo até que o vetor fique ordenado (ou quantas vezes quanto o número de elementos do array).

Ordenação

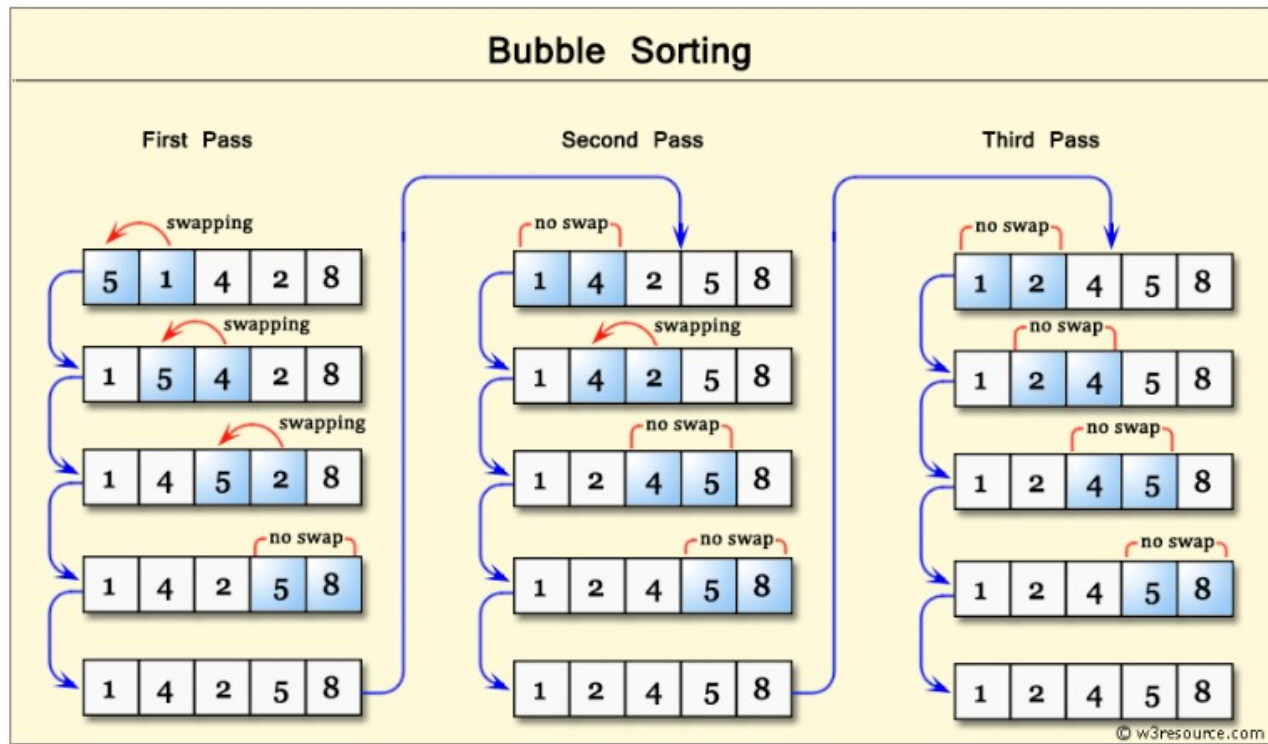


Bubble sort

val[0]	val[1]	val[2]	val[3]	val[4]	val[5]	val[6]	val[7]	
70	25	34	15	5	60	3	55	Iteração 1
25	34	15	5	60	3	55	70	Iteração 2
25	15	5	34	3	55	60	70	Iteração 3
15	5	25	3	34	55	60	70	Iteração 4
5	15	3	25	34	55	60	70	Iteração 5
5	3	15	25	34	55	60	70	Iteração 6
3	5	15	25	34	55	60	70	Iteração 7
3	5	15	25	34	55	60	70	

Ordenação

Bubble sort





Bubble sort

```
#include <stdio.h>

int bubbleSort(int val[], int qtd) {
    int i=0, x=0, aux=0;
    for(x=0; x<qtd; x++) {
        for (i=0; i<qtd-1; i++) {
            if( val[i]>val[i+1]) {
                aux=val[i];
                val[i]=val[i+1];
                val[i+1]=aux;
            }
        }
    }
    return 0;
}
```

O primeiro loop foré responsável por repetir todo o processo de ordenação do array. Dentro dele, o segundo loop for percorre o array de valores, comparando cada valor com o próximo.

Se o valor atual for maior do que o próximo valor, então ocorre uma troca entre eles, utilizando a variável "aux" como uma variável temporária para armazenar o valor atual enquanto ocorre a troca.

Esse processo de percorrer todo o array e comparar valores é repetido diversas vezes, até que não haja mais trocas a serem realizadas, o que significa que o array está totalmente ordenado.



Bubble sort

```
#include <stdio.h>

int bubbleSort(int val[], int qtd) {
    int i=0, x=0, aux=0;
    for(x=0; x<qtd; x++) {
        for (i=0; i<qtd-1; i++) {
            if( val[i]>val[i+1]) {
                aux=val[i];
                val[i]=val[i+1];
                val[i+1]=aux;
            }
        }
    }
    return 0;
}
```

```
int main() {
    int vect[10], i=0;
    printf("introduza 10 numeros inteiros:");
    for(i=0; i<10; i++) {
        scanf("%d",&vect[i]);
    }
    bubbleSort(vect,10);
    printf("Numero ordenados:\n");
    for(i=0; i<10; i++) {
        printf("%d\n",vect[i]);
    }
    return 0;
}
```



Bubble sort

```
#include <stdio.h>
#include <string.h>

void bubbleSort(char nomes[][100], int qtd) {
    int x=0,j=0;
    char temp[100];
    for (x=0; x < qtd; x++) {
        for (j=0; j < qtd-1 ; j++) {
            if (strcmp(nomes[j],nomes[j+1]) > 0)
            {
                strcpy(temp,nomes[j]);
                strcpy(nomes[j],nomes[j+1]);
                strcpy(nomes[j+1],temp);
            }
        }
    }
}
```

```
int main() {
    char nomes[10][100]; //vector de 10 strings
    int i=0;
    for(i=0; i < 10;i++) {
        printf("Introduza o nome de uma capital europeia:\n");
        fgets(nomes[i], 100, stdin);
    }
    bubbleSort(nomes,10);
    printf("Nomes das capitais ordenados:\n");
    for(i=0; i < 10; i++) {
        printf("%s\n",nomes[i]);
    }
    return 0;
}
```



Bubble sort

Para a ordenação bubble, o número de comparações é sempre o mesmo porque os dois ciclos serão repetidos um número determinado de vezes, estando ou não a lista inicialmente ordenada. Para um vetor com n elementos, o algoritmo de bubble sort executa sempre $n-1$ passagens pelos elementos do vetor.

O algoritmo de bubble sort apresenta uma variante que evita a execução de todas as passagens ($n-1$) dos elementos do vetor sempre que este fique ordenado prematuramente.

Para evitar que o processo continue mesmo depois do vetor estar ordenado, o algoritmo bubble sort modificado (ou otimizado) interrompe o processo quando houver uma passagem inteira sem trocas.

Para isso usa uma variável para verificar se existe ou não trocas, como se pode ver no diapositivo seguinte.



Bubble sort

```
#include <stdio.h>

int bubble_opt(int v[], int qtd) {
    int i, trocou=0, aux=0;
    do {
        qtd--;
        trocou = 0;
        for(i=0; i<qtd; i++) {
            if( v[i]>v[i+1]) {
                aux=v[i];
                v[i]=v[i+1];
                v[i+1]=aux;
                trocou = 1;
            }
        }
    } while(trocou==1);
    return 0;
}

int main() {
    int vect[20], temp=0, i;
    printf("Digite 20 numeros inteiros:");
    for(i=0; i<20; i++) {
        scanf("%d",&vect[i]);
    }
    bubble_opt(vect, 20);
    printf("Vector ordenado:\n");
    for(i=0; i<20; i++) {
        printf("Na posição %d fica o valor %d\n",i+1,vect[i]);
    }
    return 0;
}
```



Selection sort

O algoritmo selection Sort ou ordenação por seleção, consiste em:

Seleciona-se repetidamente o menor elemento do vetor e coloca-o na sua posição correta dentro do futuro vetor ordenado.

Ou seja, o algoritmo selection Sort:

- Seleciona, na primeira iteração, o elemento de menor valor e troca-o com o primeiro elemento.
- Repete o processo para os $N-1$ elementos restantes: o elemento com o menor valor é encontrado e trocado com o segundo elemento, e assim sucessivamente até aos dois últimos elementos.

Ordenação



Selection sort



Mínimo

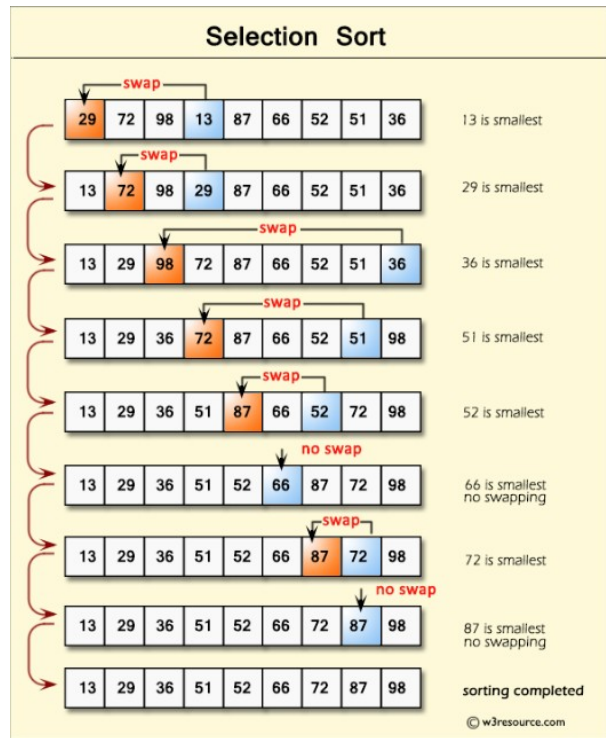


Valores já
ordenados

val[0]	val[1]	val[2]	val[3]	val[4]	val[5]	val[6]	val[7]
70	25	34	15	5	60	3	55
3	25	34	15	5	60	70	55
3	5	34	15	25	60	70	55
3	5	15	34	25	60	70	55
3	5	15	25	34	60	70	55
3	5	15	25	34	60	70	55
3	5	15	25	34	55	70	60
3	5	15	25	34	55	60	70



Selection sort



Durante a aplicação deste algoritmo, um vetor de N elementos fica decomposto em dois (sub)vetores: um que contém os elementos já ordenados e outro que contém os restantes, ainda não ordenados.

De início o (sub)vetor ordenado estará vazio, o outro (sub)vetor conterá os elementos na ordem original.

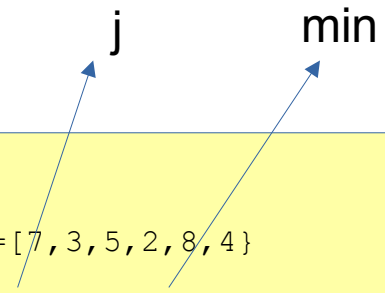
No final, o (sub)vetor ordenado conterá $N-1$ elementos ordenados, o outro (sub)vetor conterá apenas um valor.



Selection sort

```
#include <stdio.h>

int selectionSort(int v[], int qtd) {
    int i=0, j=0, min=0, aux=0;
    for(i=0; i<qtd-1; i++) {
        // procura a posição do menor valor
        min=i;
        for (j=i; j<qtd; j++) {
            if(v[j]<v[min])
                min=j;
        }
        // troca: coloca-o na posição correta
        if(min!=i){
            aux=v[i];
            v[i]=v[min];
            v[min]=aux;
        }
    }
    return 0;
}
```



V=[7,3,5,2,8,4], qtde=6
Para (i=0; i<5: i++) {
 IteraçãoE 1: i=0; min=0, v=[7,3,5,2,8,4]
 Para (j=0; j<6; j++) {
 IteraçãoI 1: i=0; j=0, v[0]=7, v[0]=7, min=0
 IteraçãoI 2: i=0; j=1, v[1]=3, v[0]=7, min=1
 IteraçãoI 3: i=0; j=2, v[2]=5, v[1]=3, min=1
 IteraçãoI 4: i=0; j=3, v[3]=2, v[1]=3, min=3
 IteraçãoI 5: i=0; j=4, v[4]=8, v[3]=2, min=3
 IteraçãoI 6: i=0; j=5, v[5]=4, v[3]=2, min=3
 }
 IteraçãoE 1: i=0; min=3, aux=7, v[0]=2, v[3]=7
 IteraçãoE 2: i=1; min=1, v=[2,3,5,7,8,4]
 ...



Selection sort

```
#include <stdio.h>

int selectionSort(int v[], int qtd) {
    int i=0, j=0, min=0, aux=0;
    for(i=0; i<qtd-1; i++) {
        // procura a posição do menor valor
        min=i;
        for (j=i; j<qtd; j++) {
            if(v[j]<v[min])
                min=j;
        }
        // troca: coloca-o na posição correta
        if(min!=i){
            aux=v[i];
            v[i]=v[min];
            v[min]=aux;
        }
    }
    return 0;
}
```

```
int main() {
    int vect[10], i=0;
    printf("introduza 10 numeros inteiros:");
    for(i=0; i<10; i++) {
        scanf("%d",&vect[i]);
    }
    selectionSort(vect,10);
    printf("Numero ordenados:\n");
    for(i=0; i<10; i++) {
        printf("%d\n",vect[i]);
    }
    return 0;
}
```



Insertion sort

O algoritmo Insertion sort ou ordenação por inserção usa a seguinte esquema de ordenação:

- Ordena inicialmente os dois primeiros elementos do array.
- De seguida, é inserido o terceiro elemento na posição ordenada em relação aos dois primeiros elementos.
- O quarto elemento é inserido na lista dos três elementos e assim sucessivamente até que todos os elementos tenham sido ordenados.



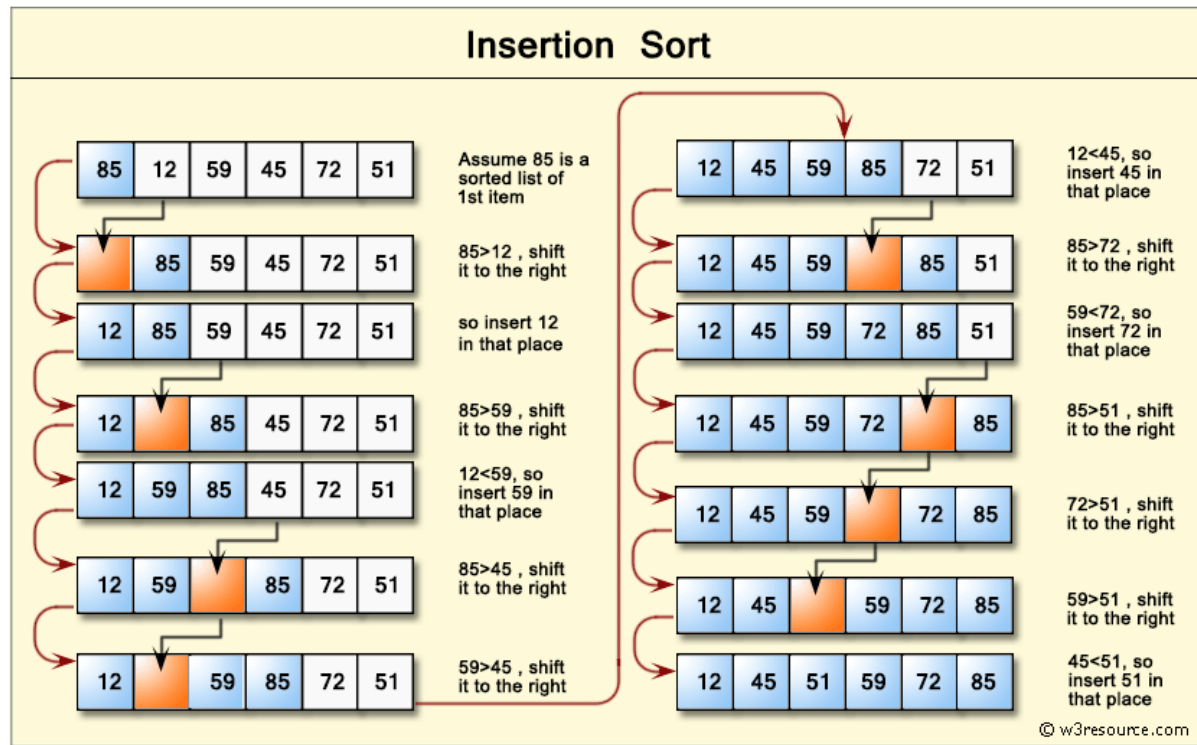
Insertion sort

 Valor a tratar
 Valores já ordenados

Insertion sort – ordem crescente

val[0]	val[1]	val [2]	val[3]	val[4]	val [5]	val[6]	val[7]
70	25	34	15	5	60	3	55
25	70	34	15	5	60	3	55
25	34	70	15	5	60	3	55
15	25	34	70	5	60	3	55
5	15	25	34	70	60	3	55
5	15	25	34	60	70	3	55
3	5	15	25	34	60	70	55
3	5	15	25	34	55	60	70

Insertion sort





Insertion sort

```
#include <stdio.h>

void insertionSort(int val[], int tam) {
    int i=0, j=0, aux=0;
    for(i=1;i<tam;i++) {
        aux = val[i];
        j = i - 1;
        while(j >= 0 && val[j]>aux) {
            val[j+1] = val[j];
            j--;
        }
        val[j+1] = aux;
    }
}
```

No loop while, a partir de j , cada elemento anterior é comparado com o valor atual (aux). Se o valor do elemento anterior for maior que o valor atual, o elemento anterior é movido para a posição seguinte ($val[j+1] = val[j]$).

O loop while continua até que um elemento menor ou igual a aux seja encontrado ou j seja menor que 0.

O valor de aux é então inserido na posição correta no array ($val[j+1] = aux$).

O loop for continua iterando sobre os elementos do array até que todos os elementos tenham sido inseridos em suas posições corretas.



Insertion sort

```
#include <stdio.h>

void insertionSort(int val[], int tam) {
    int i=0, j=0, aux=0;
    for(i=1;i<tam;i++) {
        aux = val[i];
        j = i - 1;
        while(j >= 0 && val[j]>aux) {
            val[j+1] = val[j];
            j--;
        }
        val[j+1] = aux;
    }
}
```

```
int main() {
    int vect[10], i=0;
    printf("introduza 10 numeros inteiros:");
    for(i=0; i<10; i++) {
        scanf("%d",&vect[i]);
    }
    insertionSort(vect,10);
    printf("Numero ordenados:\n");
    for(i=0; i<10; i++) {
        printf("%d\n",vect[i]);
    }
    return 0;
}
```



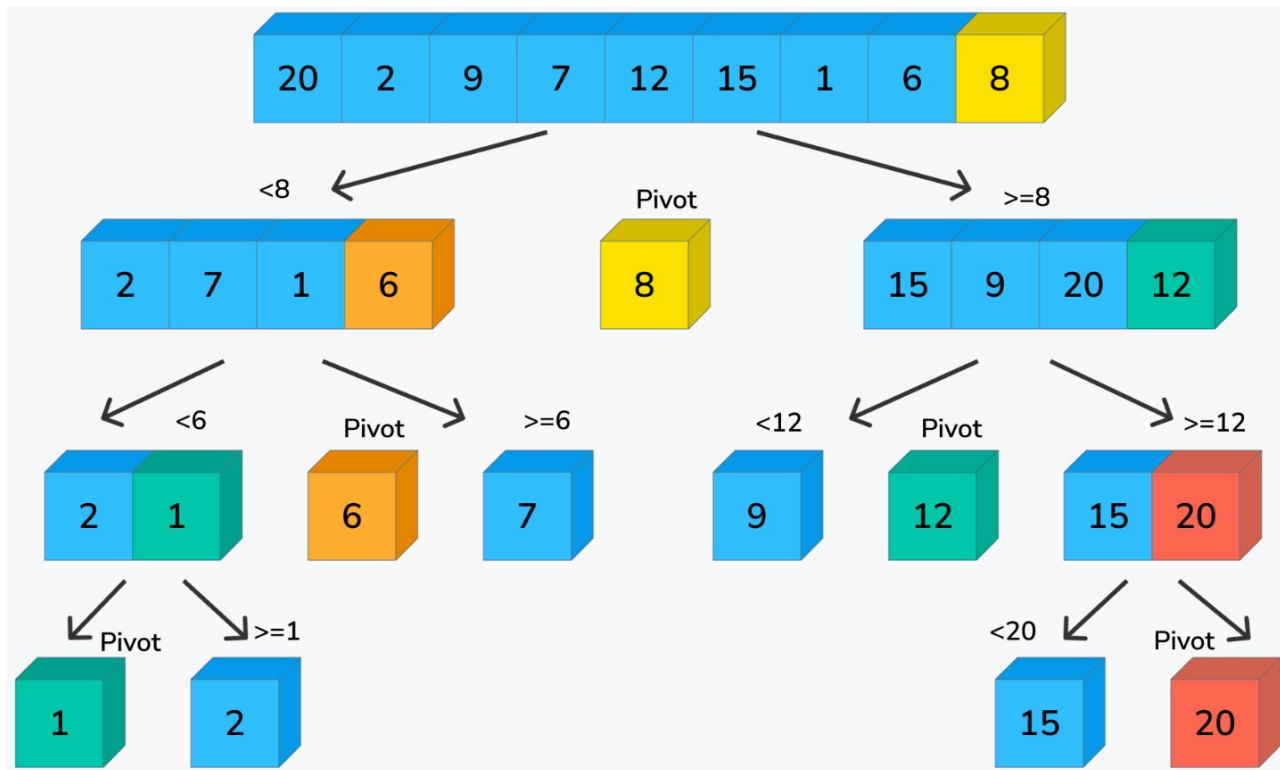
Quick sort

O algoritmo Quick Sort ou ordenação rápida, tal como o bubble sort é algoritmo de ordenação por troca. A estratégia de funcionamento é a seguinte:

- Seleciona um elemento **pivot**, normalmente o último, o primeiro ou o elemento do meio do vetor;
- Coloca o elemento selecionado para pivot na sua posição correta, ou seja, a sua posição definitiva no vetor.
- Coloca à esquerda do elemento pivot os elementos menores que este, e à direita os elementos maiores que este. Assim, supondo que o elemento pivot fica na posição x , todos os elementos entre 0 e $x-1$ são menores que o pivot e todos os elementos entre $x+1$ e n (n° de elementos do vetor) são maiores que o pivot.
- Em seguida repete-se o procedimento para cada uma das partes do vetor (esquerda e direita), usando para isso a recursividade.

Ordenação

Quick sort





Quick sort

```
#include <stdio.h>

void quickSort(int v[], int left, int right)
{
    int i,j,p=0,aux=0;
    i = left; j = right;
    p = v[(left+right)/2];
    do {
        while(v[i] < p && i < right) { i++; }
        while(p<v[j] && j > left) { j--; }
        if (i<=j) { //troca
            aux = v[i];
            v[i] = v[j];
            v[j] = aux;
            i++;
            j--;
        }
    } while (i <= j);
    if(left < j) quickSort(v, left, j);
    if(i < right) quickSort(v, i, right);
}
```

```
v=[9,2,7,6,3,8,10,1,5,4], left=0, right=9
i=0, j=9, p=v[4]=3
Faça1 {
    Faça (v[0] < 3 && i < 9) i++ → i=0
    Faça (3 < v[j] && j > 0) j-- → j=7
    Se 0<=7 {
        aux=v[0]=9, v[0]=v[7]=1, v[7]=aux=9
        i=1, j=6;
    }
} Enquanto (i<=j) // v=[1,2,7,6,3,8,10,9,5,4]
Faça2 {
    Faça (v[i] < 3 && i < 9) i++ → i=2
    Faça (3 < v[j] && j > 0) j-- → j=4
    Se 2<=4 {
        aux=v[2]=7, v[2]=v[4]=3, v[4]=aux=7
        i=3, j=3;
    }
} Enquanto (i<=j) // v=[1,2,3,6,7,8,10,9,5,4]
```



Quick sort

```
#include <stdio.h>

void quickSort(int v[], int left, int right)
{
    int i,j,p=0,aux=0;
    i = left; j = right;
    p = v[(left+right)/2];
    do {
        while(v[i] < p && i < right) { i++; }
        while(p<v[j] && j > left) { j--; }
        if (i<=j) { //troca
            aux = v[i];
            v[i] = v[j];
            v[j] = aux;
            i++;
            j--;
        }
    } while (i <= j);
    if(left < j) quickSort(v, left, j);
    if(i < right) quickSort(v, i, right);
}
```

```
int main() {
    int vect[10], i=0;
    printf("introduza 10 numeros inteiros:");
    for(i=0; i<10; i++) {
        scanf("%d",&vect[i]);
    }
    quickSort(vect,0,len(vect)-1);
    printf("Numero ordenados:\n");
    for(i=0; i<10; i++) {
        printf("%d\n",vect[i]);
    }
    return 0;
}
```



Sabendo-se que a operação de pesquisa de um elemento é uma operação muito frequente nas aplicações informáticas, é de extrema importância a escolha da estratégia mais eficiente para a efetuar, principalmente quando envolve um grande volume de dados.

Para uma primeira abordagem considera-se que os dados armazenados estão desordenados.

Posteriormente, serão consideradas estruturas com dados ordenados.



Pesquisa sequencial linear

val[0]	val[1]	val[2]	val[3]	val[4]	val[5]	val[6]	val[7]
89	22	40	15	20	5	33	7

É = 15?

Existe na posição 4



Pesquisa sequencial linear

É a forma mais simples e óbvia de pesquisa num vetor e consiste em percorrer o vetor, elemento a elemento, verificando se o valor objeto de procura é igual ao elemento do vetor.

Se chegar ao fim do vetor sem encontrar o elemento é sinal que o elemento não existe.

O numero máximo de iterações é o numero de elementos do vetor.

```
#include <stdio.h>

int pesquisaSeq(int v[], int tam, int x) {
    int i;
    for(i=0;i<tam;i++) {
        if (v[i] == x) { return(i+1); }
    }
    return -1;
}
```




Pesquisa sequencial linear

```
#include <stdio.h>

int pesquisaSeq(int v[], int tam, int x) {
    int i;
    for(i=0;i<tam;i++) {
        if (v[i] == x) { return(i+1); }
    }
    return -1;
}

int main() {
    int vect[20],i, aux=0, elem;
    printf("Digite 20 numeros inteiros:");
    for(i=0; i<20; i++) {
        scanf("%d",&vect[i]);
    }
    printf("Introduza o elemento a procurar:\n");
    scanf("%d",&elem);
    aux=pesquisaSeq(vect,20,elem);
    if (aux==-1) {
        printf("Não encontrou o elementos\n");
    }
    else {
        printf("O elemento encontra-se na posição %d\n", aux);
    }
    return 0;
}
```



Pesquisa linear ordenada

Se os elementos do vetor estiverem ordenados, é possível melhorar o algoritmo de pesquisa.

Se o elemento a procurar não existir no vetor, e este estiver ordenado por ordem crescente, logo que se encontre um elemento maior pode-se terminar a procura, evitando assim percorrer todo o vetor.

Para estes casos, o algoritmo que se apresenta a seguir é mais eficiente.



Pesquisa sequencial linear

val[0]	val[1]	val[2]	val[3]	val[4]	val[5]	val[6]	val[7]	
2	5	10	15	20	22	33	40	É = 17?

Pesquisa linear ordenada

val[0]	val[1]	val[2]	val[3]	val[4]	val[5]	val[6]	val[7]	
2	5	10	15	20	22	33	40	É = 17?

Não existe



Pesquisa linear ordenada

```
#include <stdio.h>
```

```
int bubbleSort(int val[], int qtd) {  
    int i=0, x=0, aux=0;  
    for(x=0; x<qtd; x++) {  
        for (i=0; i<qtd-1; i++) {  
            if( val[i]>val[i+1]) {  
                aux=val[i];  
                val[i]=val[i+1];  
                val[i+1]=aux;  
            }  
        }  
    }  
    return 0;  
}
```

```
int pesquisaOrdenada(int v[], int tam, int x) {  
    int i;  
    for(i=0; i<tam && v[i]<=x; i++) {  
        if (v[i] == x) {  
            return(i+1);  
        }  
    }  
    return -1;  
}
```

```
int main() {  
    int vect[20], i, aux=0, elem;  
    printf("Digite 20 numeros inteiros:");  
    for(i=0; i<20; i++) {  
        scanf("%d",&vect[i]);  
    }  
    printf("Introduza o elemento a procurar:\n");  
    scanf("%d",&elem);  
    bubbleSort(vect,20);  
    aux=pesquisaOrdenada(vect,20,elem);  
    if (aux== -1) {  
        printf("Não encontrou o elementos\n");  
    }  
    else {  
        printf("O elemento encontra-se na posição %d\n", aux);  
    }  
    return 0;  
}
```



Pesquisa binária

Nos casos em que os elementos do vetor estão ordenados, é possível aplicar um algoritmo bem mais eficiente para efetuar a pesquisa, denominado de algoritmo de pesquisa binária.

Este algoritmo usa a seguinte estratégia:

Divide o vetor a meio. Se o elemento a procurar for o elemento do meio, então está encontrado, termina a procura.

Senão,

Se o elemento a procurar é menor que elemento do meio, então procura no sub-vetor da direita, senão procura no sub-vetor da esquerda.



Pesquisa binária

val[0]	val[1]	val[2]	val[3]	val[4]	val[5]	val[6]	val[7]
2	5	10	15	20	22	33	40

É o 15?
É maior
ou menor?

2	5	10	15	20	22	33	40
---	---	----	----	----	----	----	----

É o 22?
É maior
ou menor?

2	5	10	15	20	22	33	40
---	---	----	----	----	----	----	----

É o 33?

Sim --- 33!!



Pesquisa binária

```
#include <stdio.h>

int pesquisaBin(int v[], int tam, int x) {
    //fim fica com o último índice
    int meio=0, ini=0, fim=tam-1;
    while (ini <= fim) {
        meio=(fim+ini)/2;
        if (x==v[meio]) {
            return (meio+1);
        }
        if (x < v[meio]) {
            // pesquisa na parte esquerda
            fim=meio-1;
        }
        else {
            // pesquisa na parte direita
            ini=meio+1;
        }
    }
    return -1;
}

int main() {
    int vect[18]={3,5,7,9,23,45,67,89,90,120, 122,
124,128,130, 140,150,155,160};
    int aux=0, elem;
    printf("Introduza o elemento a procurar\n");
    scanf("%d",&elem);
    aux=pesquisaBin(vect,18,elem);
    if (aux!=-1) {
        printf("Não encontrou o elemento\n");
    }
    else {
        printf("O elemento encontra-se na posição %d\n", aux);
    }
    return 0;
}
```



Pesquisa binária recursiva

```
#include <stdio.h>

int pesquisaBinRec(int v[], int ini, int fim, int x) {
    int meio=0;
    if (ini>fim) return -1;
    meio=(ini+fim)/2;
    if (x==v[meio]) {
        // encontrou o elemento a pesquisar
        return (meio+1);
    }
    if (x < v[meio]) {
        // pesquisa na parte direita
        pesquisaBinRec (v, ini, meio-1, x);
    }
    else {
        // pesquisa na parte esquerda
        pesquisaBinRec (v, meio+1, fim, x);
    }
}
```

```
int main() {
    int vect[18]={3,5,7,9,23,45,67,89,90,120, 122,
124,128,130, 140,150,155,160};
    int aux=0, elem;
    printf("Introduza o elemento a procurar\n");
    scanf("%d",&elem);
    aux=pesquisaBinRec(vect,0,17,elem);
    if (aux!=-1) {
        printf("Não encontrou o elemento\n");
    }
    else {
        printf("O elemento encontra-se na posição %d\n", aux);
    }
    return 0;
}
```




Pesquisa binária

O numero de comparações efetuadas por este algoritmo é substancialmente menor que nos anteriormente apresentados, visto que cada iteração reduz para metade o conjunto de valores a pesquisar. Alguns dados:

Num conjunto de 100 elementos são necessárias, no máximo, 6 interações;

Num conjunto de 10 000 elementos são necessárias, no máximo, 13 interações;

Num conjunto de 1 000 000 elementos são necessárias, no máximo, 19 interações;

Num conjunto de 1 000 000 000 elementos são necessárias, no máximo, 29 interações;



Exercícios

1. Implemente uma função que recebe como parâmetro um array de nomes e a respetiva quantidade e ordena alfabeticamente os nomes.
2. Implemente o algoritmos de pesquisa binária, que verifica se um determinado nome recebido como parâmetro existe no array de nomes.
3. Implemente um programa que receba do utilizador os nomes, invoque a função de ordenação criada anteriormente e apresenta o resultado para o ecrã.
4. Depois de ordenar os nomes, deve pedir ao utilizador um nome e, recorrendo ao algoritmo de pesquisa binária, verificar se o nome existe no array.

o teu • de partida



Instituto Politécnico
de Viana do Castelo

www.ipvc.pt